

ShopCart Course Project Documentation

Mini E-Commerce Product Catalog & Cart (React.js)

Project: ShopCart | **Type:** React Single-Page Application

Course: React Course Assignment

Repository: github.com/rupak-at/react_project_college

Live Demo: react-project-college.vercel.app

1. Project Overview and Objective

ShopCart is a lightweight single-page e-commerce storefront built entirely with functional React components. The objective of this project is to demonstrate hands-on application of the core React concepts covered in the course, namely component-based architecture, props, state management with hooks, event handling, conditional and list rendering, client-side routing, and persistence.

Users can browse a mock product catalog, search and filter products, sort the results, add items to a cart, adjust quantities, remove items, and place a simulated order. The cart is stored in `localStorage` so it survives page refreshes, and the interface is fully responsive across desktop and mobile widths.

2. Modules / Features Implemented

2.1 Modules

- **Product Catalog Module:** displays 16 local mock products across Electronics, Clothing, and Accessories, rendered as a responsive card grid.
- **Search, Filter & Sort Module:** live search by name, category filter (segmented control), and sorting by price or name.
- **Cart Management Module:** add to cart with automatic quantity merging, update quantities, remove items, running totals and live item-count badge.
- **Checkout Module:** order summary panel and a simulated order-confirmation state with toast notification.

- **Persistence Module:** custom `useLocalStorage` hook syncing cart state with the browser's `localStorage`.
- **Routing Module:** React Router views for Catalog (/) and Cart (/cart).

2.2 Feature List

- Product catalog with `.map()` list rendering and unique keys
- Search by product name (controlled form input)
- Category filter with Apple-style segmented control
- Sort by price (low to high / high to low) and name (A to Z)
- Add to cart; repeated adds increment quantity
- Quantity controls (increase and decrease) and item removal
- Running order total and live cart-count badge in the navbar
- Toast notifications for add / remove / order-placed events
- Cart persistence via `localStorage` (survives refresh)
- Empty-cart, no-results, and order-confirmation states
- Responsive layout for desktop and mobile (Tailwind utilities)

2.3 Component Structure

Component / Hook	Responsibility
<code>App.jsx</code>	Router setup, cart state wiring, toast container
<code>Navbar</code>	Brand link and cart navigation with live count badge
<code>FilterBar</code>	Search input, category segmented control, sort select
<code>ProductGrid</code>	Responsive grid, maps products into cards
<code>ProductCard</code>	Single product tile with image, price, Add button
<code>CartItem</code>	Cart line with quantity controls and remove button
<code>CartSummary</code>	Order totals and checkout button
<code>EmptyState</code>	Reusable empty / confirmation state panel

useCart Custom hook: cart state + add, change, remove, clear handlers

useLocalStorage Custom hook: state synced to localStorage via useEffect

CatalogPage / CartPage Route-level view components

3. Technologies / Tools Used

Tool	Purpose
React 19	Functional components and hooks
Vite 8	Build tool and dev server
React Router DOM 7	Client-side routing (/, /cart)
Tailwind CSS 4	Utility-first responsive styling
React Hot Toast	Toast notifications
localStorage API	Cart persistence across refreshes
Git / GitHub	Version control and public repository
Vercel	Deployment and SPA hosting
Oxlint	Linting

3.1 Setup / Run Instructions

1. git clone https://github.com/rupak-at/react_project_college.git
2. npm install
3. npm run dev, then open <http://localhost:5173>
4. npm run build for a production build; npm run lint for linting.

4. Screenshots

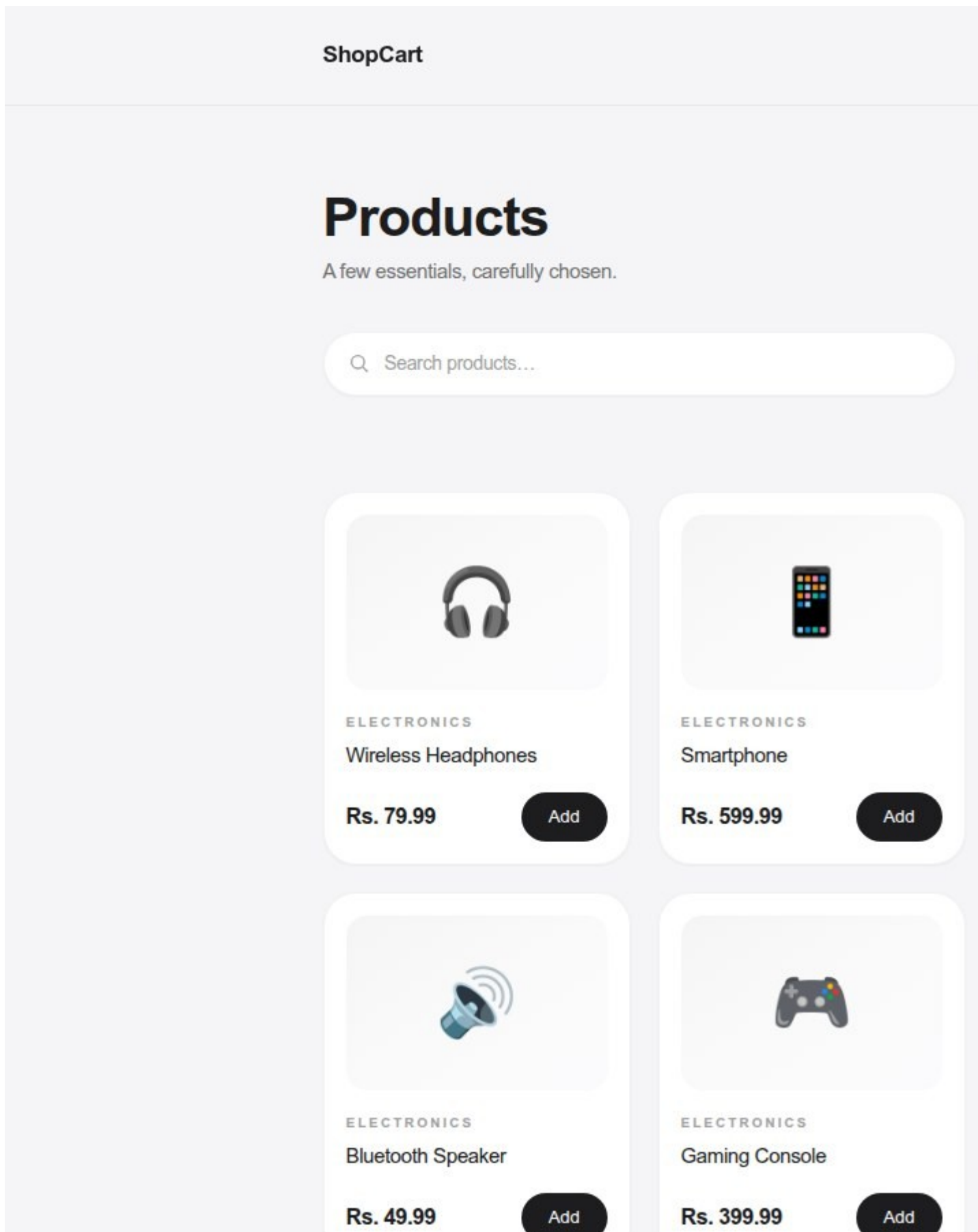


Fig 1: Product catalog (desktop): navbar, search bar, segmented category filter, and product grid.

Your Cart

5 items ready for checkout.



Smartphone

Rs. 599.99 each

- 1 +



Laptop

Rs. 999.99 each

- 1 +



T-Shirt

Rs. 19.99 each

- 2 +



Sunglasses

Rs. 35.99 each

- 1 +

Fig 2: Cart page (desktop): cart items with quantity controls, running total, and order summary.

Products

A few essentials, carefully chosen.

🔍 Search products...

All

Electronics

Clothing

Accessories

Sort by



16 products



Fig 3: Catalog view on a mobile viewport showing the responsive two-column layout.

5. Appendix

5.1 References

- React documentation (react.dev)
- React Router documentation (reactrouter.com)
- Tailwind CSS documentation (tailwindcss.com)
- Vite guide (vite.dev)
- React Hot Toast (react-hot-toast.com)

5.2 Git History (Abridged)

1. Initialize Vite React project with sample store name
2. Add mock product catalog data
3. Add reusable UI components
4. Add useCart hook with cart handlers
5. Add catalog page with search, filter and sort and wire routing
6. Add cart page with quantity controls and checkout
7. Persist cart in localStorage
8. Replace custom CSS with Tailwind utility classes
9. Add toast notifications for cart actions
10. Refactor to clean premium UI with rupee pricing
11. Add app screenshots to README
12. Add Vercel SPA rewrite and live demo link

5.3 Extra Notes & Known Limitations

- Checkout is simulated: there is no real payment processing or order history.
- Product artwork uses emoji tiles rather than real product photography.
- Prices are shown in rupees (Rs.) for display purposes.
- No user accounts; cart is device-local via localStorage.
- Cart contents persist indefinitely (no expiry).